

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Noor Muhammad Akmal Sulaiman NIM. 2410817210007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Muhammad Akmal Sulaiman
NIM : 2410817210007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom., M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	9
Jetpack Compose.....	9
XML.....	26
B. Output Program.....	45
Jetpack Compose.....	45
XML.....	48
C. Pembahasan.....	51
Jetpack Compose.....	51
XML.....	55
SOAL 2	60
A. Pembahasan.....	60
SOAL 3	62
A. Pembahasan.....	62
TAUTAN GIT.....	63

DAFTAR GAMBAR

Gambar 1. Contoh UI List.....	7
Gambar 2. Contoh UI Detail	8
Gambar 3. Contoh Penerapan LazyRow/Carousel.....	8
Gambar 4. Screenshot Tampilan Awal Soal 1 Compose	46
Gambar 5. Screensho Tampilant Detail Soal 1 Compose	46
Gambar 6. Screenshot Pengaturan Bahasa Soal 1 Compose.....	47
Gambar 7. Screenshot Tampilan Bahasa Indonesia Soal 1 Compose.....	47
Gambar 8. Screenshot Tampilan Landscape Soal 1 Compose.....	48
Gambar 9. Screenshot Tampilan Awal Soal 1 XML	49
Gambar 10. Screenshot Tampilan Detail Jawaban Soal 1 XML	49
Gambar 11. Screenshot Pengaturan Bahasa Soal 1 XML.....	50
Gambar 12. Screenshot Tampilan Bahasa Indonesia Soal 1 XML	50
Gambar 13. Screenshot Tampilan Landscape Soal 1 XML.....	51

DAFTAR TABEL

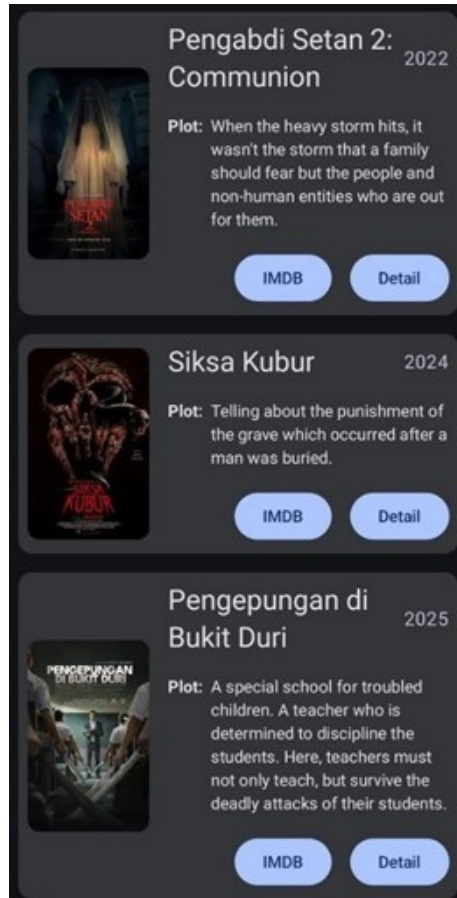
Tabel 1. Source Code MainActivity.kt Compose	9
Tabel 2. Source Code Game.kt Compose	9
Tabel 3. Source Code GameRepository.kt Compose.....	10
Tabel 4. Source Code MainViewModel.kt Compose	11
Tabel 5. Source Code HomeScreen.kt Compose	11
Tabel 6. Source Code DetailScreen.kt Compose	17
Tabel 7. Source Code SettinsgScreen.kt Compose	20
Tabel 8. Source Code Screen.kt Compose	24
Tabel 9. Source Code NavGraph.kt Compose	24
Tabel 10. Source Code MainActivity.kt XML.....	26
Tabel 11. Source Code Game.kt XML	26
Tabel 12. Source Code GameRepository.kt XML	27
Tabel 13. Source Code MainViewModel.kt XML.....	28
Tabel 14. Source Code HomeFragment.kt XML	29
Tabel 15. Source Code DetailFragment.kt XML	31
Tabel 16. Source Code SettingsFragment.kt XML	33
Tabel 17. Source Code GameAdapter.kt.xml XML	34
Tabel 18. Source Code CarouselAdapter.kt XML	36
Tabel 19. Source Code activity_main.xml XML	36
Tabel 20. Source Code fragment_home.xml XML.....	37
Tabel 21. Source Code fragment_detail.xml XML.....	39
Tabel 22. Source Code fragment_settings.xml XML	40
Tabel 23. Source Code item_game_vertical.xml XML	41
Tabel 24. Source Code item_game_carousel.xml XML.....	43
Tabel 25. Source Code nav_graph.xml XML	44

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:
 - Item bisa bergeser secara finite atau infinite loop sebanyak jumlah item yang ada pada scrollable-list.

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



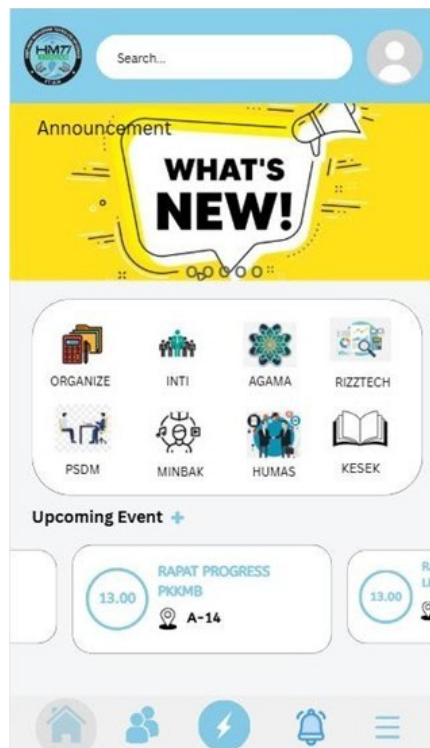
Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5' kembali ke item awal '1')



Gambar 3. Contoh Penerapan LazyRow/Carousel

A. Source Code

Jetpack Compose

MainActivity.kt

Tabel 1. Source Code MainActivity.kt Compose

```
1 package io.github.orizynpx.fivegamescompose
2
3 import android.os.Bundle
4 import androidx.activity.compose.setContent
5 import androidx.activity.enableEdgeToEdge
6 import androidx.appcompat.app.AppCompatActivity
7 import
8     io.github.orizynpx.fivegamescompose.ui.navigation.FiveGamesNavGraph
9 import
10    io.github.orizynpx.fivegamescompose.ui.theme.FiveGamesComposeTheme
11
12 class MainActivity : AppCompatActivity() {
13     override fun onCreate(savedInstanceState: Bundle?) {
14         super.onCreate(savedInstanceState)
15         enableEdgeToEdge()
16         setContent {
17             FiveGamesComposeTheme {
18                 FiveGamesNavGraph()
19             }
20         }
21     }
22 }
```

Game.kt

Tabel 2. Source Code Game.kt Compose

```
1 package io.github.orizynpx.fivegamescompose.data.model
2
3 import androidx.annotation.DrawableRes
4 import androidx.annotation.StringRes
5
6 data class Game(
7     val id: Int,
8     @DrawableRes val imageResourceId: Int,
9     @StringRes val titleResourceId: Int,
10    @StringRes val yearResourceId: Int,
11    @StringRes val genreResourceId: Int,
12    @StringRes val detailResourceId: Int,
13    @StringRes val urlResourceId: Int
14 )
```

GameRepository.kt

Tabel 3. Source Code GameRepository.kt Compose

```
1 package io.github.orizynpx.fivegamescompose.data
2
3 import io.github.orizynpx.fivegamescompose.R
4 import
5 io.github.orizynpx.fivegamescompose.data.model.Game
6
7 class GameRepository {
8     fun getGames(): List<Game> {
9         return listOf(
10             Game(
11                 0,
12                 R.drawable.bullet_ant,
13                 R.string.bullet_ant,
14                 R.string.year_2024,
15                 R.string.bullet_ant_genre,
16                 R.string.bullet_ant_detail,
17                 R.string.bullet_ant_url
18             ),
19             Game(
20                 1,
21                 R.drawable.wizard,
22                 R.string.wizard,
23                 R.string.year_2025,
24                 R.string.wizard_genre,
25                 R.string.wizard_detail,
26                 R.string.wizard_url
27             ),
28             Game(
29                 2,
30                 R.drawable.ammid,
31                 R.string.ammid,
32                 R.string.year_2025,
33                 R.string.ammid_genre,
34                 R.string.ammid_detail,
35                 R.string.ammid_url
36             ),
37             Game(
38                 3,
39                 R.drawable.robert,
40                 R.string.robert,
41                 R.string.year_2025,
42                 R.string.robert_genre,
43                 R.string.robert_detail,
44                 R.string.robert_url
45             )
46         )
47     }
48 }
```

45	Game (
46	4,
47	R.drawable.personal_space,
48	R.string.personal_space,
49	R.string.year_2025,
50	R.string.personal_space_genre,
51	R.string.personal_space_detail,
52	R.string.personal_space_url
53	)
54	)
55	}
56	}
57	package io.github.orizynpx.fivegamescompose.data
58	
59	import io.github.orizynpx.fivegamescompose.R
60	import
	io.github.orizynpx.fivegamescompose.data.model.Game

MainViewModel.kt

Tabel 4. Source Code MainViewModel.kt Compose

1	package io.github.orizynpx.fivegamescompose.ui.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import
	io.github.orizynpx.fivegamescompose.data.GameRepository
5	import
	io.github.orizynpx.fivegamescompose.data.model.Game
6	import kotlinx.coroutines.flow.MutableStateFlow
7	import kotlinx.coroutines.flow.StateFlow
8	import kotlinx.coroutines.flow.asStateFlow
9	
10	class MainViewModel : ViewModel() {
11	private val repository = GameRepository()
12	
13	private val _games =
	MutableStateFlow<List<Game>>(emptyList())
14	val games: StateFlow<List<Game>> =
	_games.asStateFlow()
15	
16	init {
17	_games.value = repository.getGames()
18	}
19	}

HomeScreen.kt

Tabel 5. Source Code HomeScreen.kt Compose

```

1 package io.github.orizynpx.fivegamescompose.ui.home
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.clickable
7 import androidx.compose.foundation.layout.Arrangement
8 import androidx.compose.foundation.layout.Box
9 import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.PaddingValues
11 import androidx.compose.foundation.layout.Row
12 import androidx.compose.foundation.layout.Spacer
13 import androidx.compose.foundation.layout.aspectRatio
14 import androidx.compose.foundation.layout.fillMaxSize
15 import androidx.compose.foundation.layout.fillMaxWidth
16 import androidx.compose.foundation.layout.height
17 import androidx.compose.foundation.layout.padding
18 import androidx.compose.foundation.layout.width
19 import androidx.compose.foundation.lazy.LazyColumn
20 import androidx.compose.foundation.lazy.LazyRow
21 import androidx.compose.foundation.lazy.items
22 import
23     androidx.compose.foundation.shape.RoundedCornerShape
24 import androidx.compose.material.icons.Icons
25 import androidx.compose.material.icons.filled.Settings
26 import androidx.compose.material3.Button
27 import androidx.compose.material3.Card
28 import androidx.compose.material3.CardDefaults
29 import
30     androidx.compose.material3.ExperimentalMaterial3Api
31 import androidx.compose.material3.Icon
32 import androidx.compose.material3.IconButton
33 import androidx.compose.material3.MaterialTheme
34 import androidx.compose.material3.OutlinedButton
35 import androidx.compose.material3.Scaffold
36 import androidx.compose.material3.Text
37 import androidx.compose.material3.TopAppBar
38 import androidx.compose.material3.TopAppBarDefaults
39 import androidx.compose.runtime.Composable
40 import androidx.compose.runtime.collectAsState
41 import androidx.compose.runtime.getValue
42 import androidx.compose.ui.Alignment
43 import androidx.compose.ui.Modifier
44 import androidx.compose.ui.draw.clip
45 import androidx.compose.ui.layout.ContentScale
46 import androidx.compose.ui.platform.LocalContext
47 import androidx.compose.ui.res.painterResource
48 import androidx.compose.ui.res.stringResource
49 import androidx.compose.ui.text.font.FontWeight

```

```

48 import androidx.compose.ui.text.style.TextOverflow
49 import androidx.compose.ui.unit.dp
50 import androidx.compose.ui.unit.sp
51 import androidx.lifecycle.viewmodel.compose.viewModel
52 import io.github.orizynpx.fivegamescompose.R
53 import
54   io.github.orizynpx.fivegamescompose.data.model.Game
55 import
56   io.github.orizynpx.fivegamescompose.ui.viewmodel.MainView
57   wModel
58
59 @OptIn(ExperimentalMaterial3Api::class)
60 @Composable
61 fun HomeScreen(
62     onNavigateToDetail: (Int) -> Unit,
63     onNavigateToSettings: () -> Unit,
64     viewModel: MainViewModel = viewModel()
65 ) {
66     val games by viewModel.games.collectAsState()
67
68     Scaffold(
69         topBar = {
70             TopAppBar(
71                 title = {
72                     Text(
73                         text =
74                         stringResource(R.string.app_name),
75                         style =
76                         MaterialTheme.typography.titleLarge
77                     )
78                 },
79                 actions = {
80                     IconButton(onClick =
81                         onNavigateToSettings) {
82                         Icon(
83                             imageVector =
84                             Icons.Default.Settings,
85                             contentDescription =
86                             stringResource(R.string.title_settings)
87                         )
88                     }
89                 },
90                 colors =
91                 TopAppBarDefaults.topAppBarColors(
92                     containerColor =
93                     MaterialTheme.colorScheme.surface
94                 )
95             )
96         }
97     )
98 }

```

```

87     ) { innerPadding ->
88         LazyColumn(
89             modifier = Modifier
90                 .fillMaxSize()
91                 .padding(innerPadding),
92             contentPadding = PaddingValues(bottom =
16.dp)
93         ) {
94             // Highlight / Carousel section
95             item {
96                 Text(
97                     text =
stringResource(R.string.tv_highlight),
98                     fontSize = 20.sp,
99                     fontWeight = FontWeight.Bold,
100                     modifier = Modifier.padding(16.dp)
101                 )
102             }
103             item {
104                 GameCarousel(
105                     games = games,
106                     onGameClick = {
onNavigateToDetail(it.id) }
107                 )
108             }
109
110             // Game list section
111             item {
112                 Text(
113                     text =
stringResource(R.string.tv_game_list),
114                     fontSize = 20.sp,
115                     fontWeight = FontWeight.Bold,
116                     modifier = Modifier.padding(16.dp)
117                 )
118             }
119             items(games, key = { it.id }) { game ->
120                 GameListItem(
121                     game = game,
122                     onDetailClick = {
onNavigateToDetail(game.id) },
123                 )
124             }
125         }
126     }
127 }
128
129 @Composable
130 fun GameCarousel(

```

```

131     games: List<Game>,
132     onGameClick: (Game) -> Unit
133 ) {
134     LazyRow(
135         contentPadding = PaddingValues(horizontal =
136         16.dp),
137         horizontalArrangement =
138         Arrangement.spacedBy(8.dp)
139     ) {
140         items(games, key = { it.id }) { game ->
141             Box(
142                 modifier = Modifier
143                     .height(200.dp)
144                     .aspectRatio(16f / 9f)
145                     .clip(RoundedCornerShape(16.dp))
146                     .clickable { onGameClick(game) }
147             ) {
148                 Image(
149                     painter = painterResource(id =
150                     game.imageResourceId), contentDescription =
151                     stringResource(game.titleResourceId),
152                     contentScale = ContentScale.Crop,
153                     modifier = Modifier.fillMaxSize()
154                 )
155             }
156         }
157     }
158 }
159
160 @Composable
161 fun GameListItem(
162     game: Game,
163     onDetailClick: () -> Unit,
164 ) {
165     val context = LocalContext.current
166     val urlString = stringResource(game.urlResourceId)
167
168     Card(
169         modifier = Modifier
170             .fillMaxWidth()
171             .padding(horizontal = 8.dp, vertical =
172             4.dp),
173         shape = RoundedCornerShape(12.dp),
174         elevation =
175         CardDefaults.cardElevation(defaultElevation = 4.dp)
176     ) {
177         Row(
178             modifier = Modifier
179                 .fillMaxWidth()

```

```

174         .padding(8.dp),
175         verticalAlignment = Alignment.Top
176     ) {
177         Image(
178             painter = painterResource(id =
game.imageResourceId),
179             contentDescription =
stringResource(game.titleResourceId),
180             contentScale = ContentScale.Crop,
181             modifier = Modifier
182                 .width(100.dp)
183                 .height(140.dp)
184                 .clip(RoundedCornerShape(8.dp))
185         )
186
187         Spacer(modifier = Modifier.width(12.dp))
188
189         Column(
190             modifier = Modifier.weight(1f)
191         ) {
192             Row(
193                 modifier = Modifier.fillMaxWidth(),
194                 horizontalArrangement =
Arrangement.SpaceBetween,
195                 verticalAlignment = Alignment.Top
196             ) {
197                 Text(
198                     text =
stringResource(game.titleResourceId),
199                     fontSize = 18.sp,
200                     fontWeight = FontWeight.Bold,
201                     modifier = Modifier.weight(1f),
202                     maxLines = 3,
203                     overflow = TextOverflow.Ellipsis
204                 )
205                 Spacer(modifier =
Modifier.width(8.dp))
206                 Text(
207                     text =
stringResource(game.yearResourceId),
208                     fontSize = 12.sp,
209                     color =
MaterialTheme.colorScheme.onSurfaceVariant
210                 )
211             }
212
213             Spacer(modifier = Modifier.height(4.dp))
214
215             Row {

```

```

216         Text(
217             text =
218             stringResource(R.string.tv_genre_label),
219             fontSize = 14.sp,
220             fontWeight = FontWeight.Bold
221         )
222         Spacer(modifier =
223             Modifier.width(4.dp))
224         Text(
225             text =
226             stringResource(game.genreResourceId),
227             fontSize = 14.sp
228         )
229     }
230     Spacer(modifier = Modifier.height(8.dp))
231     Row(
232         horizontalArrangement =
233         Arrangement.End,
234         modifier = Modifier.fillMaxWidth()
235     ) {
236         OutlinedButton(onClick =
237             onDetailClick) {
238             Text(text =
239             stringResource(R.string.btn_detail))
240         }
241         Spacer(modifier =
242             Modifier.width(8.dp))
243         Button(
244             onClick = {
245                 context.startActivity(
246                     Intent(Intent.ACTION_VIEW, Uri.parse(urlString))
247                 )
248             }
249         ) {
250             Text(text =
251             stringResource(R.string.btn_link))
252         }
253     }
254 }

```

Class DetailScreen.kt

Tabel 6. Source Code DetailScreen.kt Compose

```

1 package io.github.orizynpx.fivegamescompose.ui.detail
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.Column
5 import androidx.compose.foundation.layout.fillMaxSize
6 import androidx.compose.foundation.layout.fillMaxWidth
7 import androidx.compose.foundation.layout.padding
8 import
  androidx.compose.foundation.layout.wrapContentHeight
9 import androidx.compose.foundation.rememberScrollState
10 import androidx.compose.foundation.verticalScroll
11 import androidx.compose.material.icons.Icons
12 import
  androidx.compose.material.icons.automirrored.filled.Arro
  wBack
13 import
  androidx.compose.material3.ExperimentalMaterial3Api
14 import androidx.compose.material3.Icon
15 import androidx.compose.material3.IconButton
16 import androidx.compose.material3.MaterialTheme
17 import androidx.compose.material3.Scaffold
18 import androidx.compose.material3.Text
19 import androidx.compose.material3.TopAppBar
20 import androidx.compose.material3.TopAppBarDefaults
21 import androidx.compose.runtime.Composable
22 import androidx.compose.runtime.collectAsState
23 import androidx.compose.runtime.getValue
24 import androidx.compose.ui.Modifier
25 import androidx.compose.ui.layout.ContentScale
26 import androidx.compose.ui.res.painterResource
27 import androidx.compose.ui.res.stringResource
28 import androidx.compose.ui.text.font.FontWeight
29 import androidx.compose.ui.unit.dp
30 import androidx.compose.ui.unit.sp
31 import androidx.lifecycle.viewmodel.compose.viewModel
32 import io.github.orizynpx.fivegamescompose.R
33 import
  io.github.orizynpx.fivegamescompose.ui.viewmodel.MainVie
  wModel
34
35 @OptIn(ExperimentalMaterial3Api::class)
36 @Composable
37 fun DetailScreen(
38     gameId: Int,
39     onNavigateBack: () -> Unit,
40     viewModel: MainViewModel = viewModel()
41 ) {
42     val games by viewModel.games.collectAsState()
43     val game = games.find { it.id == gameId }

```

```

44
45     Scaffold(
46         topBar = {
47             TopAppBar(
48                 title = { Text(text =
stringResource(R.string.btn_detail)) },
49                 navigationIcon = {
50                     IconButton(onClick = onNavigateBack)
51 {
52                     Icon(
53                         imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
54                         contentDescription = "Back"
55                     )
56                 },
57                 colors =
TopAppBarDefaults.topAppBarColors(
58                     containerColor =
MaterialTheme.colorScheme.surface
59                 )
60             )
61         }
62     ) { innerPadding ->
63         game?.let {
64             Column(
65                 modifier = Modifier
66                     .fillMaxSize()
67                     .padding(innerPadding)
68
69                 .verticalScroll(rememberScrollState())
70             ) {
71                 Image(
72                     painter = painterResource(id =
it.imageResourceId),
73                     contentDescription =
stringResource(it.titleResourceId),
74                     contentScale = ContentScale.Crop,
75                     modifier = Modifier
76                         .fillMaxWidth()
77                         .wrapContentHeight()
78                 )
79                 Text(
80                     text =
stringResource(it.titleResourceId),
81                     fontSize = 24.sp,
82                     fontWeight = FontWeight.Bold,
83                     modifier = Modifier.padding(16.dp)

```

84	)
85	
86	Text (
87	text =
	stringResource(it.detailResourceId),
88	fontSize = 16.sp,
89	modifier =
	Modifier.padding(horizontal = 16.dp)
90	)
91	}
92	}
93	}
94	}
95	package io.github.orizynpx.fivegamescompose.ui.detail
96	
97	import androidx.compose.foundation.Image
98	import androidx.compose.foundation.layout.Column
99	import androidx.compose.foundation.layout.fillMaxSize
100	import androidx.compose.foundation.layout.fillMaxWidth

Class SettingsScreen.kt

Tabel 7. Source Code SettingsScreen.kt Compose

1	package io.github.orizynpx.fivegamescompose.ui.settings
2	
3	import androidx.appcompat.app.AppCompatActivityDelegate
4	import androidx.compose.foundation.clickable
5	import androidx.compose.foundation.layout.Column
6	import androidx.compose.foundation.layout.Row
7	import androidx.compose.foundation.layout.Spacer
8	import androidx.compose.foundation.layout.fillMaxSize
9	import androidx.compose.foundation.layout.fillMaxWidth
10	import androidx.compose.foundation.layout.padding
11	import androidx.compose.foundation.layout.width
12	import androidx.compose.material.icons.Icons
13	import
	androidx.compose.material.icons.automirrored.filled.Arro
	wBack
14	import androidx.compose.material3.AlertDialog
15	import
	androidx.compose.material3.ExperimentalMaterial3Api
16	import androidx.compose.material3.HorizontalDivider
17	import androidx.compose.material3.Icon
18	import androidx.compose.material3.IconButton
19	import androidx.compose.material3.MaterialTheme
20	import androidx.compose.material3.RadioButton
21	import androidx.compose.material3.Scaffold
22	import androidx.compose.material3.Text

```

23 import androidx.compose.material3.TextButton
24 import androidx.compose.material3.TopAppBar
25 import androidx.compose.material3.TopAppBarDefaults
26 import androidx.compose.runtime.Composable
27 import androidx.compose.runtime.getValue
28 import androidx.compose.runtime.mutableStateOf
29 import androidx.compose.runtime.remember
30 import androidx.compose.runtime.setValue
31 import androidx.compose.ui.Alignment
32 import androidx.compose.ui.Modifier
33 import androidx.compose.ui.res.stringResource
34 import androidx.compose.ui.text.font.FontWeight
35 import androidx.compose.ui.unit.dp
36 import androidx.compose.ui.unit.sp
37 import androidx.core.os.LocaleListCompat
38 import io.github.orizynpx.fivegamescompose.R
39
40 data class LanguageOption(val name: String, val code:
    String)
41
42 @OptIn(ExperimentalMaterial3Api::class)
43 @Composable
44 fun SettingsScreen(
45     onNavigateBack: () -> Unit
46 ) {
47     val languageOptions = listOf(
48         LanguageOption(
49             name =
50             stringResource(R.string.language_english),
51             code = "en"
52         ),
53         LanguageOption(
54             name =
55             stringResource(R.string.language_indonesian),
56             code = "id"
57         )
58     )
59     var showLanguageDialog by remember {
60         mutableStateOf(false) }
61     val currentLocale =
62         AppCompatActivity.getApplicationLocales()
63     val currentCode = if (currentLocale.isEmpty) "en"
64     else currentLocale[0]?.language ?: "en"
65     var selectedCode by remember {
66         mutableStateOf(currentCode) }
67
68     Scaffold(
69         topBar = {

```

```

65         TopAppBar(
66             title = { Text(text =
stringResource(R.string.title_settings)) },
67             navigationIcon = {
68                 IconButton(onClick = onNavigateBack)
69             {
70                 Icon(
71                     imageVector =
Icons.AutoMirrored.Filled.ArrowBack,
72                     contentDescription = "Back"
73                 )
74             },
75             colors =
TopAppBarDefaults.topAppBarColors(
76                 containerColor =
MaterialTheme.colorScheme.surface
77             )
78         )
79     }
80     ) { innerPadding ->
81         Column(
82             modifier = Modifier
83                 .fillMaxSize()
84                 .padding(innerPadding)
85         ) {
86             // Category header
87             Text(
88                 text =
stringResource(R.string.pref_category_language_settings)
89             ,
90                 fontSize = 12.sp,
91                 fontWeight = FontWeight.Medium,
92                 color =
MaterialTheme.colorScheme.primary,
93                 modifier = Modifier.padding(start =
16.dp, top = 16.dp, bottom = 8.dp)
94             )
95             // Language preference row
96             val currentLangName = languageOptions.find {
it.code == selectedCode }?.name
97             ?: languageOptions.first().name
98
99             Column(
100                 modifier = Modifier
101                     .fillMaxWidth()
102                     .clickable { showLanguageDialog =
true }

```

```

103         .padding(horizontal = 16.dp,
vertical = 12.dp)
104     ) {
105         Text(
106             text =
stringResource(R.string.pref_title_change_language),
107             fontSize = 16.sp,
108             fontWeight = FontWeight.Normal
109         )
110         Text(
111             text = currentLangName,
112             fontSize = 14.sp,
113             color =
MaterialTheme.colorScheme.onSurfaceVariant
114         )
115     }
116
117     HorizontalDivider(modifier =
Modifier.padding(horizontal = 16.dp))
118 }
119 }
120
121     if (showLanguageDialog) {
122         AlertDialog(
123             onDismissRequest = { showLanguageDialog =
false },
124             title = { Text(text =
stringResource(R.string.pref_title_change_language)) },
125             text = {
126                 Column {
127                     languageOptions.forEach { option ->
128                         Row(
129                             modifier = Modifier
130                                 .fillMaxWidth()
131                                 .clickable {
132                                     selectedCode =
option.code
133                                     showLanguageDialog =
false
134                                     val appLocale =
LocaleListCompat.forLanguageTags(option.code)
135                                     AppCompatActivity.setApplicationLocales(appLocale)
136                                 }
137                                 .padding(vertical =
4.dp),
138                                     verticalAlignment =
Alignment.CenterVertically
139             ) {

```

140	RadioButton(
141	selected = selectedCode
	== option.code,
142	onClick = {
143	selectedCode =
	option.code
144	showLanguageDialog =
	false
145	val appLocale =
	LocaleListCompat.forLanguageTags(option.code)
146	AppCompatActivity.setApplicationLocales(appLocale)
147	}
148	)
149	Spacer(modifier =
	Modifier.width(8.dp))
150	Text(text = option.name)
151	}
152	}
153	}
154	},
155	confirmButton = {
156	TextButton(onClick = {
	showLanguageDialog = false }) {
157	Text(text = "Cancel")
158	}
159	}
160	)
161	}
162	}

Screen.kt

Tabel 8. Source Code Screen.kt Compose

1	package
	io.github.orizynpx.fivegamescompose.ui.navigation
2	
3	sealed class Screen(val route: String) {
4	data object Home : Screen("home")
5	data object Detail : Screen("detail/{gameId}") {
6	fun createRoute(gameId: Int) = "detail/\$gameId"
7	}
8	data object Settings : Screen("settings")
9	}

Class NavGraph.kt

Tabel 9. Source Code NavGraph.kt Compose

```

1 package
  io.github.orizynpx.fivegamescompose.ui.navigation
2
3 import androidx.compose.runtime.Composable
4 import androidx.navigation.NavHostController
5 import androidx.navigation.NavType
6 import androidx.navigation.compose.NavHost
7 import androidx.navigation.compose.composable
8 import androidx.navigation.compose.rememberNavController
9 import androidx.navigation.navArgument
10 import
  io.github.orizynpx.fivegamescompose.ui.navigation.Screen
11 import
  io.github.orizynpx.fivegamescompose.ui.detail.DetailScreen
12 import
  io.github.orizynpx.fivegamescompose.ui.home.HomeScreen
13 import
  io.github.orizynpx.fivegamescompose.ui.settings.SettingsScreen
14
15 @Composable
16 fun FiveGamesNavGraph(
17     navController: NavHostController =
rememberNavController()
18 ) {
19     NavHost(
20         navController = navController,
21         startDestination = Screen.Home.route
22     ) {
23         composable(Screen.Home.route) {
24             HomeScreen(
25                 onNavigateToDetail = { gameId ->
26
navController.navigate(Screen.Detail.createRoute(gameId)
27
                ),
28                 onNavigateToSettings = {
29
navController.navigate(Screen.Settings.route)
30
                }
31             )
32         }
33
34         composable(
35             route = Screen.Detail.route,
36             arguments = listOf(
37                 navArgument("gameId") { type =
NavType.IntType }

```

38	)
39	) { backStackEntry ->
40	val gameId =
	backStackEntry.arguments?.getInt("gameId") ?: -1
41	DetailScreen(
42	gameId = gameId,
43	onNavigateBack = {
	navController.navigateUp() }
44	)
45	}
46	
47	composable(Screen.Settings.route) {
48	SettingsScreen(
49	onNavigateBack = {
	navController.navigateUp() }
50	)
51	}
52	}
53	}

XML

MainActivity.kt

Tabel 10. Source Code MainActivity.kt XML

10	package io.github.orizynpx.fivegamesxml
11	
12	import android.os.Bundle
13	import androidx.activity.enableEdgeToEdge
14	import androidx.appcompat.app.AppCompatActivity
15	import
	io.github.orizynpx.fivegamesxml.databinding.ActivityMain
	Binding
16	
17	class MainActivity : AppCompatActivity() {
18	override fun onCreate(savedInstanceState: Bundle?) {
19	super.onCreate(savedInstanceState)
20	enableEdgeToEdge()
21	val binding =
	ActivityMainBinding.inflate(layoutInflater)
22	setContentView(binding.root)
23	}
24	}

Game.kt

Tabel 11. Source Code Game.kt XML

```

1 package io.github.orizynpx.fivegamesxml.data.model
2
3 import androidx.annotation.DrawableRes
4 import androidx.annotation.StringRes
5
6 data class Game(
7     val id: Int,
8     @DrawableRes val imageResourceId: Int,
9     @StringRes val titleResourceId: Int,
10    @StringRes val yearResourceId: Int,
11    @StringRes val genreResourceId: Int,
12    @StringRes val detailResourceId: Int,
13    @StringRes val url: Int
14 )

```

GameRepository.kt

Tabel 12. Source Code GameRepository.kt XML

```

1 package io.github.orizynpx.fivegamesxml.data
2
3 import io.github.orizynpx.fivegamesxml.R
4 import io.github.orizynpx.fivegamesxml.data.model.Game
5
6 class GameRepository {
7     fun getGames(): List<Game> {
8         return listOf(
9             Game(
10                0,
11                R.drawable.bullet_ant,
12                R.string.bullet_ant,
13                R.string.year_2024,
14                R.string.bullet_ant_genre,
15                R.string.bullet_ant_detail,
16                R.string.bullet_ant_url
17            ),
18            Game(
19                1,
20                R.drawable.wizard,
21                R.string.wizard,
22                R.string.year_2025,
23                R.string.wizard_genre,
24                R.string.wizard_detail,
25                R.string.wizard_url
26            ),
27            Game(
28                2,
29                R.drawable.ammid,
30                R.string.ammid,

```

```

31         R.string.year_2025,
32         R.string.ammid_genre,
33         R.string.ammid_detail,
34         R.string.ammid_url
35     ),
36     Game(
37         3,
38         R.drawable.robert,
39         R.string.robert,
40         R.string.year_2025,
41         R.string.robert_genre,
42         R.string.robert_detail,
43         R.string.robert_url
44     ),
45     Game(
46         4,
47         R.drawable.personal_space,
48         R.string.personal_space,
49         R.string.year_2025,
50         R.string.personal_space_genre,
51         R.string.personal_space_detail,
52         R.string.personal_space_url
53     )
54 )
55 }
56 }

```

MainViewModel.kt

Tabel 13. Source Code MainViewModel.kt XML

```

1 package io.github.orizynpx.fivegamesxml.ui
2
3 import androidx.lifecycle.LiveData
4 import androidx.lifecycle.MutableLiveData
5 import androidx.lifecycle.ViewModel
6 import
7 io.github.orizynpx.fivegamesxml.data.GameRepository
8 import io.github.orizynpx.fivegamesxml.data.model.Game
9
10 class MainViewModel : ViewModel() {
11     private val repository = GameRepository()
12     private val _games = MutableLiveData<List<Game>>()
13     val games: LiveData<List<Game>> = _games
14
15     init {
16         _games.value = repository.getGames()
17     }
18 }

```

HomeFragment.kt

Tabel 14. Source Code HomeFragment.kt XML

1	package io.github.orizynpx.fivegamesxml.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.navigation.fragment.findNavController
12	import io.github.orizynpx.fivegamesxml.R
13	import androidx.recyclerview.widget.LinearLayoutManager
14	import androidx.recyclerview.widget.RecyclerView
15	import
	com.google.android.material.carousel.CarouselLayoutManager
16	import
	com.google.android.material.carousel.CarouselSnapHelper
17	import
	com.google.android.material.carousel.HeroCarouselStrategy
18	import
	io.github.orizynpx.fivegamesxml.databinding.FragmentHomeBinding
19	import
	io.github.orizynpx.fivegamesxml.ui.adapter.CarouselAdapter
20	import
	io.github.orizynpx.fivegamesxml.ui.adapter.GameAdapter
21	
22	class HomeFragment : Fragment() {
23	
24	private var _binding: FragmentHomeBinding? = null
25	private val binding get() = _binding!!
26	
27	private val viewModel: MainViewModel by viewModels()
28	
29	private var gameAdapter: GameAdapter? = null
30	private var carouselAdapter: CarouselAdapter? = null
31	
32	override fun onCreateView(
33	inflater: LayoutInflater,
34	container: ViewGroup?,

```

35         savedInstanceState: Bundle?
36     ): View {
37         _binding = FragmentHomeBinding.inflate(inflater,
38         container, false)
39         return binding.root
40     }
41     override fun onViewCreated(view: View,
42     savedInstanceState: Bundle?) {
43         super.onViewCreated(view, savedInstanceState)
44         binding.btnSettings.setOnClickListener {
45             findNavController().navigate(R.id.action_homeFragment_to
46             _settingsFragment)
47         }
48         if (gameAdapter == null || carouselAdapter ==
49         null) {
50             setupAdapters()
51         }
52         setupRecyclerViews()
53         viewModel.games.observe(viewLifecycleOwner) {
54             gamesList ->
55                 gameAdapter?.submitList(gamesList)
56                 carouselAdapter?.submitList(gamesList)
57             }
58         }
59         private fun setupAdapters() {
60             gameAdapter = GameAdapter(
61                 onDetailClick = { game ->
62                 navigateToDetail(game.id) },
63                 onLinkClick = { url ->
64                 startActivity(Intent(Intent.ACTION_VIEW,
65                 Uri.parse(url)))
66             }
67             )
68             gameAdapter?.stateRestorationPolicy =
69             RecyclerView.Adapter.StateRestorationPolicy.PREVENT_WHEN
70             _EMPTY
71             carouselAdapter = CarouselAdapter { game ->
72                 navigateToDetail(game.id)
73             }
74             carouselAdapter?.stateRestorationPolicy =

```

74	RecyclerView.Adapter.StateRestorationPolicy.PREVENT_WHEN_EMPTY
75	}
76	
77	private fun setupRecyclerViews() {
78	binding.rvGames.apply {
79	layoutManager =
80	LinearLayoutManager(requireContext())
81	adapter = gameAdapter
82	setHasFixedSize(true)
83	}
84	binding.rvCarousel.apply {
85	layoutManager =
86	CarouselLayoutManager(HeroCarouselStrategy())
87	adapter = carouselAdapter
88	setHasFixedSize(true)
89	CarouselSnapHelper().attachToRecyclerView(this)
90	}
91	
92	private fun navigateToDetail(gameId: Int) {
93	val bundle = Bundle().apply { putInt("gameId",
94	gameId) }
95	findNavController().navigate(R.id.action_homeFragment_to
96	_detailFragment, bundle)
97	}
98	override fun onDestroyView() {
99	super.onDestroyView()
100	_binding = null
101	}

DetailFragment.kt

Tabel 15. Source Code DetailFragment.kt XML

1	package io.github.orizynpx.fivegamesxml.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment

```

8 import
  io.github.orizynpx.fivegamesxml.data.GameRepository
9 import
  io.github.orizynpx.fivegamesxml.databinding.FragmentDetailBinding
10
11 class DetailFragment : Fragment() {
12
13     private var _binding: FragmentDetailBinding? = null
14     private val binding get() = _binding!!
15
16     override fun onCreateView(
17         inflater: LayoutInflater, container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View {
20         _binding =
            FragmentDetailBinding.inflate(inflater, container,
            false)
21         return binding.root
22     }
23
24     override fun onViewCreated(view: View,
25         savedInstanceState: Bundle?) {
26         super.onViewCreated(view, savedInstanceState)
27
28         val gameId = arguments?.getInt("gameId") ?: -1
29         val game = GameRepository().getGames().find {
30             it.id == gameId }
31
32         binding.toolbarDetail.setNavigationOnClickListener {
33             requireActivity().onBackPressedDispatcher.onBackPressed()
34         }
35
36         game?.let {
37             binding.imgDetail.setImageResource(it.imageResourceId)
38             binding.tvDetailTitle.text =
39                 getString(it.titleResourceId)
40             binding.tvDetailDescription.text =
41                 getString(it.detailResourceId)
42         }
43
44     override fun onDestroyView() {
45         super.onDestroyView()
46         _binding = null
47     }
48 }

```

44	}
45	}

SettingsFragment.kt

Tabel 16. Source Code SettingsFragment.kt XML

1	package io.github.orizynpx.fivegamesxml.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import android.widget.FrameLayout
8	import androidx.appcompat.app.AppCompatActivityDelegate
9	import androidx.core.os.LocaleListCompat
10	import androidx.navigation.fragment.findNavController
11	import androidx.preference.ListPreference
12	import androidx.preference.PreferenceFragmentCompat
13	import io.github.orizynpx.fivegamesxml.R
14	
15	class SettingsFragment : PreferenceFragmentCompat() {
16	
17	override fun onCreatePreferences(savedInstanceState: Bundle?, rootKey: String?) {
18	
19	setPreferencesFromResource(R.xml.root_preferences,
20	rootKey)
21	
22	val languagePreference =
23	findPreference<ListPreference>("language")
24	
25	languagePreference?.setOnPreferenceChangeListener { _,
26	newValue ->
27	val code = newValue as String
28	val appLocale: LocaleListCompat =
29	LocaleListCompat.forLanguageTags(code)
30	
31	AppCompatActivity.setApplicationLocales(appLocale)
32	true
33	}
34	
35	override fun onCreateView(
36	inflator: LayoutInflater,
37	container: ViewGroup?,
38	savedInstanceState: Bundle?
39	): View {

34	val view = inflater.inflate(R.layout.fragment_settings, container, false)
35	val containerView = view.findViewById<FrameLayout>(R.id.settings_container)
36	
37	val settingsView = super.onCreateView(inflater, containerView, savedInstanceState)
38	containerView.addView(settingsView)
39	
40	return view
41	}
42	
43	override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
44	super.onViewCreated(view, savedInstanceState)
45	
46	view.findViewById<com.google.android.material.appbar.Mat erialToolbar>(R.id.toolbarSettings)
47	.setNavigationOnClickListener {
48	findNavController().navigateUp()
49	}
50	}
51	}

GameAdapter.kt

Tabel 17. Source Code GameAdapter.kt.xml XML

1	package io.github.orizynpx.fivegamesxml.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import io.github.orizynpx.fivegamesxml.data.model.Game
9	import io.github.orizynpx.fivegamesxml.databinding.ItemGameVert icalBinding
10	
11	class GameAdapter(12 private val onDetailClick: (Game) -> Unit, 13 private val onLinkClick: (String) -> Unit 14) : ListAdapter<Game, GameAdapter.GameViewHolder>(DiffCallback) { 15

```

16     class GameViewHolder(private val binding:
ItemGameVerticalBinding) :
17         RecyclerView.ViewHolder(binding.root) {
18
19         fun bind(game: Game, onDetailClick: (Game) ->
Unit, onLinkClick: (String) -> Unit) {
20             val context = binding.root.context
21
22             binding.imgGame.setImageResource(game.imageResourceId)
23             binding.tvTitle.text =
context.getString(game.titleResourceId)
24             binding.tvYear.text =
context.getString(game.yearResourceId)
25             binding.tvGenre.text =
context.getString(game.genreResourceId)
26
27             binding.btnDetail.setOnClickListener {
onDetailClick(game) }
28
29             binding.btnLink.setOnClickListener {
30                 val urlString =
context.getString(game.url)
31                 onLinkClick(urlString)
32             }
33         }
34     }
35
36     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): GameViewHolder {
37         val binding = ItemGameVerticalBinding.inflate(
38             LayoutInflater.from(parent.context), parent,
false
39         )
40         return GameViewHolder(binding)
41     }
42
43     override fun onBindViewHolder(holder:
GameViewHolder, position: Int) {
44         holder.bind(getItem(position), onDetailClick,
onLinkClick)
45     }
46
47     companion object DiffCallback :
DiffUtil.ItemCallback<Game>() {
48         override fun areItemsTheSame(oldItem: Game,
newItem: Game): Boolean = oldItem.id == newItem.id
49         override fun areContentsTheSame(oldItem: Game,
newItem: Game): Boolean = oldItem == newItem

```

50	}
51	}

CarouselAdapter.kt

Tabel 18. Source Code CarouselAdapter.kt XML

1	package io.github.orizynpx.fivegamesxml.ui.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.ListAdapter
6	import androidx.recyclerview.widget.RecyclerView
7	import io.github.orizynpx.fivegamesxml.data.model.Game
8	import
	io.github.orizynpx.fivegamesxml.databinding.ItemGameCaro
	uselBinding
9	
10	class CarouselAdapter(private val onClick: (Game) ->
	Unit) :
11	ListAdapter<Game,
	CarouselAdapter.ViewHolder>(GameAdapter.DiffCallback) {
12	
13	class ViewHolder(val binding:
	ItemGameCarouselBinding) :
	RecyclerView.ViewHolder(binding.root)
14	
15	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): ViewHolder {
16	return
	ViewHolder(ItemGameCarouselBinding.inflate(LayoutInflate
	r.from(parent.context), parent, false))
17	}
18	
19	override fun onBindViewHolder(holder: ViewHolder,
	position: Int) {
20	val game = getItem(position)
21	
	holder.binding.imgCarousel.setImageResource(game.imageRe
	sourceId)
22	holder.binding.imgCarousel.setOnClickListener {
23	onClick(game)
24	}
25	}
26	}

File activity_main.xml

Tabel 19. Source Code activity_main.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_host_fragment"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:defaultNavHost="true"
10	app:navGraph="@navigation/nav_graph" />

File fragment_home.xml

Tabel 20. Source Code fragment_home.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.core.widget.NestedScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/nsvHome"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	android:fitsSystemWindows="true"
9	tools:context=".ui.HomeFragment">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="vertical"
15	android:paddingBottom="16dp">
16	
17	
18	<com.google.android.material.appbar.MaterialToolbar
19	android:id="@+id/toolbar_home"
20	android:layout_width="match_parent"
21	android:layout_height="?attr/actionBarSize"
22	app:title="@string/app_name"
	app:titleTextAppearance="@style/TextAppearance.Material3.TitleLarge">
23	
24	<ImageButton
25	android:id="@+id/btn_settings"

```

26         android:layout_width="wrap_content"
27         android:layout_height="wrap_content"
28         android:layout_gravity="end"
29         android:layout_marginEnd="16dp"
30
31     android:background="?attr/selectableItemBackgroundBorder
less"
32
33     android:contentDescription="@string/title_settings"
34     android:src="@drawable/ic_settings"
35     app:tint="?attr/colorPrimary" />
36
37 </com.google.android.material.appbar.MaterialToolbar>
38
39     <TextView
40         android:layout_width="wrap_content"
41         android:layout_height="wrap_content"
42         android:padding="16dp"
43         android:text="@string/tv_highlight"
44         android:textSize="20sp"
45         android:textStyle="bold" />
46
47     <androidx.recyclerview.widget.RecyclerView
48         android:id="@+id/rv_carousel"
49         android:layout_width="match_parent"
50         android:layout_height="200dp"
51         android:clipChildren="false"
52         android:clipToPadding="false"
53         android:paddingHorizontal="16dp"
54         android:saveEnabled="true" />
55
56     <TextView
57         android:layout_width="wrap_content"
58         android:layout_height="wrap_content"
59         android:padding="16dp"
60         android:text="@string/tv_game_list"
61         android:textSize="20sp"
62         android:textStyle="bold" />
63
64     <androidx.recyclerview.widget.RecyclerView
65         android:id="@+id/rv_games"
66         android:layout_width="match_parent"
67         android:layout_height="wrap_content"
68         android:nestedScrollingEnabled="false"
69         tools:listitem="@layout/item_game_vertical"
70     />
71
72 </LinearLayout>

```

70	</androidx.core.widget.NestedScrollView>
----	--

File fragment_detail.xml

Tabel 21. Source Code fragment_detail.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fitsSystemWindows="true"
8	android:orientation="vertical">
9	
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_detail"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	app:navigationIcon="@drawable/ic_back"
15	app:navigationIconTint="?attr/colorPrimary"
16	app:title="@string/btn_detail" />
17	
18	<androidx.core.widget.NestedScrollView
19	xmlns:android="http://schemas.android.com/apk/res/android"
20	xmlns:app="http://schemas.android.com/apk/res-auto"
21	xmlns:tools="http://schemas.android.com/tools"
22	android:layout_width="match_parent"
23	android:layout_height="match_parent"
24	android:fitsSystemWindows="true">
25	
26	<androidx.constraintlayout.widget.ConstraintLayout
27	android:layout_width="match_parent"
28	android:layout_height="wrap_content">
29	
30	<ImageView
31	android:id="@+id/imgDetail"
32	android:layout_width="match_parent"
33	android:layout_height="wrap_content"
34	android:scaleType="centerCrop"
35	
	app:layout_constraintTop_toTopOf="parent"
36	tools:src="@drawable/bullet_ant" />

```

37
38         <TextView
39             android:id="@+id/tvDetailTitle"
40             android:layout_width="match_parent"
41             android:layout_height="wrap_content"
42             android:padding="16dp"
43             android:textSize="24sp"
44             android:textStyle="bold"
45
46 app:layout_constraintTop_toBottomOf="@id/imgDetail"
47             tools:text="Game Title" />
48
49         <TextView
50             android:id="@+id/tvDetailDescription"
51             android:layout_width="match_parent"
52             android:layout_height="wrap_content"
53             android:paddingHorizontal="16dp"
54             android:textSize="16sp"
55
56 app:layout_constraintTop_toBottomOf="@id/tvDetailTitle"
57             tools:text="This is the full description
58 of the game..." />
59
60 </androidx.constraintlayout.widget.ConstraintLayout>
61 </androidx.core.widget.NestedScrollView>
62 </LinearLayout>

```

File fragment_settings.xml

Tabel 22. Source Code fragment_settings.xml XML

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      android:fitsSystemWindows="true">
9      <com.google.android.material.appbar.MaterialToolbar
10         android:id="@+id/toolbarSettings"
11         android:layout_width="match_parent"
12         android:layout_height="?attr/actionBarSize"
13         app:navigationIcon="@drawable/ic_back"
14         app:navigationIconTint="?attr/colorPrimary"
15         app:title="@string/title_settings" />

```

16	
17	<FrameLayout
18	android:id="@+id/settings_container"
19	android:layout_width="match_parent"
20	android:layout_height="match_parent" />
21	
22	</LinearLayout>

File item_game_vertical.xml

Tabel 23. Source Code item_game_vertical.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="12dp"
9	app:cardElevation="4dp">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="8dp">
15	
16	<ImageView
17	android:id="@+id/img_game"
18	android:layout_width="100dp"
19	android:layout_height="140dp"
20	android:scaleType="centerCrop"
21	
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintTop_toTopOf="parent"
24	app:layout_constraintBottom_toBottomOf="parent"
25	app:layout_constraintVertical_bias="0.0"
26	tools:srcCompat="@drawable/bullet_ant"
27	/>
28	<TextView
29	android:id="@+id/tv_year"
30	android:layout_width="wrap_content"
31	android:layout_height="wrap_content"
	android:textSize="4dp"

```

32 android:textColor="@color/material_dynamic_neutral60"
33     app:layout_constraintEnd_toEndOf="parent"
34     app:layout_constraintTop_toTopOf="parent"
35     tools:text="2024" />
36
37     <TextView
38         android:id="@+id/tv_title"
39         android:layout_width="0dp"
40         android:layout_height="wrap_content"
41         android:layout_marginStart="12dp"
42         android:layout_marginEnd="8dp"
43         android:textSize="18sp"
44         android:textStyle="bold"
45
46 app:layout_constraintEnd_toStartOf="@id/tv_year"
47
48 app:layout_constraintStart_toEndOf="@id/img_game"
49     app:layout_constraintTop_toTopOf="parent"
50     tools:text="Game Title That Is Very Long and
51 Wraps Around" />
52
53     <LinearLayout
54         android:id="@+id/ll_genre"
55         android:layout_width="0dp"
56         android:layout_height="wrap_content"
57         android:layout_marginTop="4dp"
58         android:orientation="horizontal"
59         app:layout_constraintEnd_toEndOf="parent"
60
61 app:layout_constraintStart_toStartOf="@id/tv_title"
62
63 app:layout_constraintTop_toBottomOf="@id/tv_title">
64
65     <TextView
66         android:layout_width="wrap_content"
67         android:layout_height="wrap_content"
68         android:text="@string/tv_genre_label"
69         android:textSize="14sp"
70         android:textStyle="bold" />
71
72     <Space
73         android:layout_width="14sp"
74         android:layout_height="match_parent" />
75
76     <TextView
77         android:id="@+id/tv_genre"
78         android:layout_width="wrap_content"
79         android:layout_height="wrap_content"

```

```

75         android:textSize="14sp"
76         tools:text="Action RPG" />
77     </LinearLayout>
78
79     <LinearLayout
80         android:id="@+id/ll_buttons"
81         android:layout_width="wrap_content"
82         android:layout_height="wrap_content"
83         android:layout_marginTop="8dp"
84         android:orientation="horizontal"
85
86         app:layout_constraintBottom_toBottomOf="parent"
87         app:layout_constraintEnd_toEndOf="parent"
88
89         app:layout_constraintTop_toBottomOf="@id/ll_genre"
90         app:layout_constraintVertical_bias="1.0">
91
92         <Button
93             android:id="@+id/btn_detail"
94
95             style="?attr/materialButtonOutlinedStyle"
96             android:layout_width="wrap_content"
97             android:layout_height="wrap_content"
98             android:text="@string/btn_detail" />
99
100         <Space
101             android:layout_width="8dp"
102             android:layout_height="wrap_content" />
103
104         <Button
105             android:id="@+id/btn_link"
106             android:layout_width="wrap_content"
107             android:layout_height="wrap_content"
108             android:text="@string/btn_link" />
109     </LinearLayout>
110
111 </androidx.constraintlayout.widget.ConstraintLayout>
112 </com.google.android.material.card.MaterialCardView>

```

File item_game_carousel.xml

Tabel 24. Source Code item_game_carousel.xml XML

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <com.google.android.material.carousel.MaskableFrameLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"

```

4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="wrap_content"
6	android:layout_height="200dp"
7	android:duplicateParentState="true"
8	android:layout_marginHorizontal="8dp"
9	
	app:shapeAppearance="?attr/shapeAppearanceCornerExtraLarge">
10	
11	<ImageView
12	android:No.="@+No./img_carousel"
13	android:layout_width="match_parent"
14	android:layout_height="match_parent"
15	android:scaleType="centerCrop"
16	android:importantForAccessibility="No."
17	tools:src="@drawable/bullet_ant" />
18	
19	</com.google.android.material.carousel.MaskableFrameLayout>

File nav_graph.xml

Tabel 25. Source Code nav_graph.xml XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/homeFragment">
7	
8	<fragment
9	android:id="@+id/homeFragment"
10	
	android:name="io.github.orizynpx.fivegamesxml.ui.HomeFragment"
11	android:label="fragment_home"
12	tools:layout="@layout/fragment_home">
13	
14	<action
15	
	android:id="@+id/action_homeFragment_to_detailFragment"
16	app:destination="@id/detailFragment" />
17	
18	<action

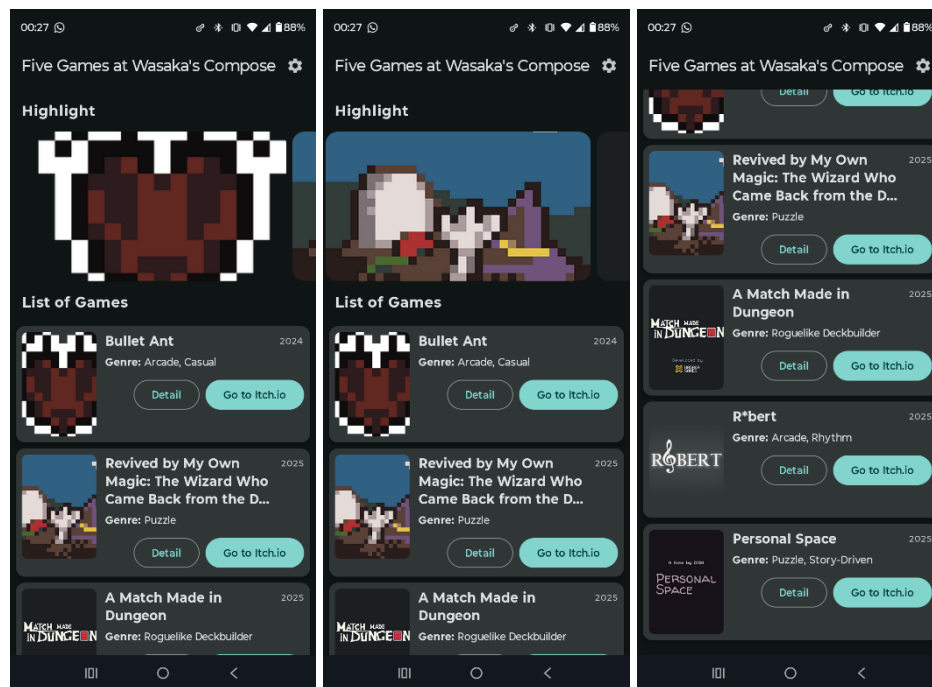
```

19     android:id="@+id/action_homeFragment_to_settingsFragment
20         app:destination="@id/settingsFragment"
21     />
22     </fragment>
23
24     <fragment
25         android:id="@+id/detailFragment"
26         android:name="io.github.orizynpx.fivegamesxml.ui.DetailF
27             ragment"
28             android:label="Game Details"
29             tools:layout="@layout/fragment_detail">
30         <argument
31             android:name="gameId"
32             app:argType="integer" />
33         </fragment>
34
35         <fragment
36             android:id="@+id/settingsFragment"
37             android:name="io.github.orizynpx.fivegamesxml.ui.Setting
38             sFragment"
39             android:label="Settings" />
40     </navigation>

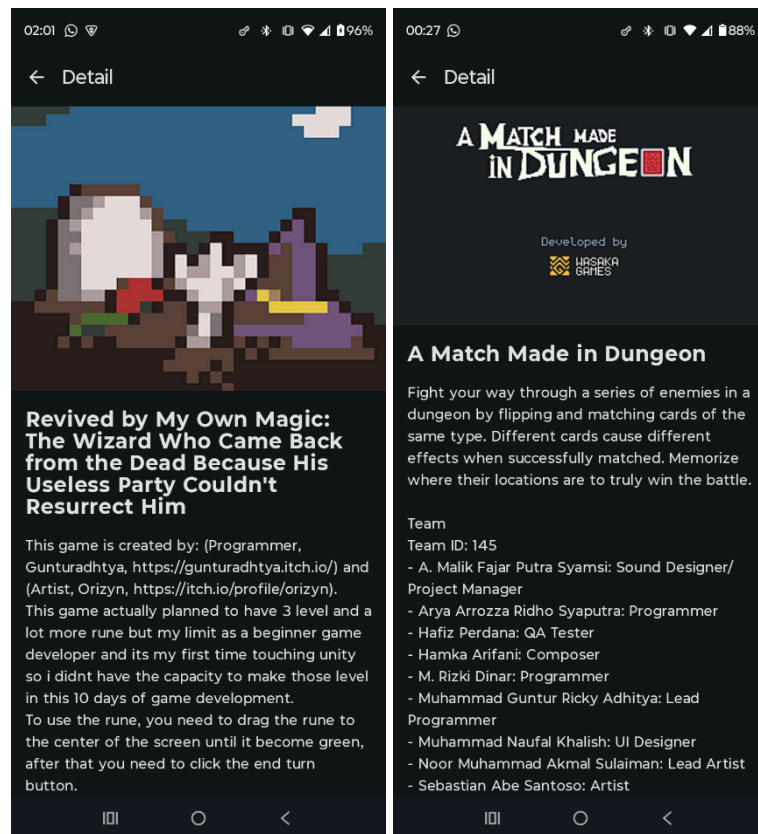
```

B. Output Program

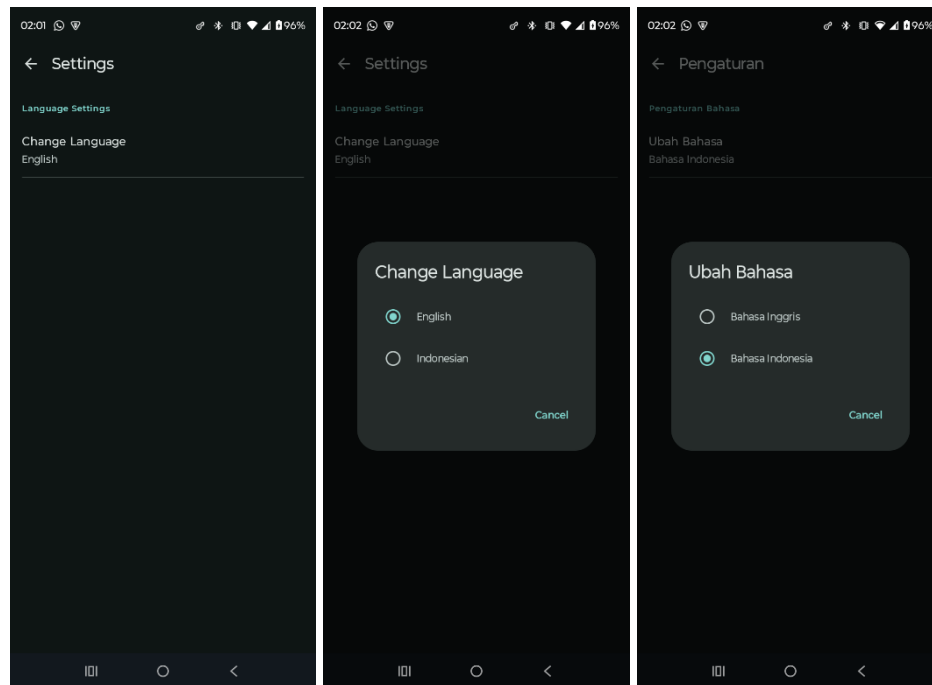
Jetpack Compose



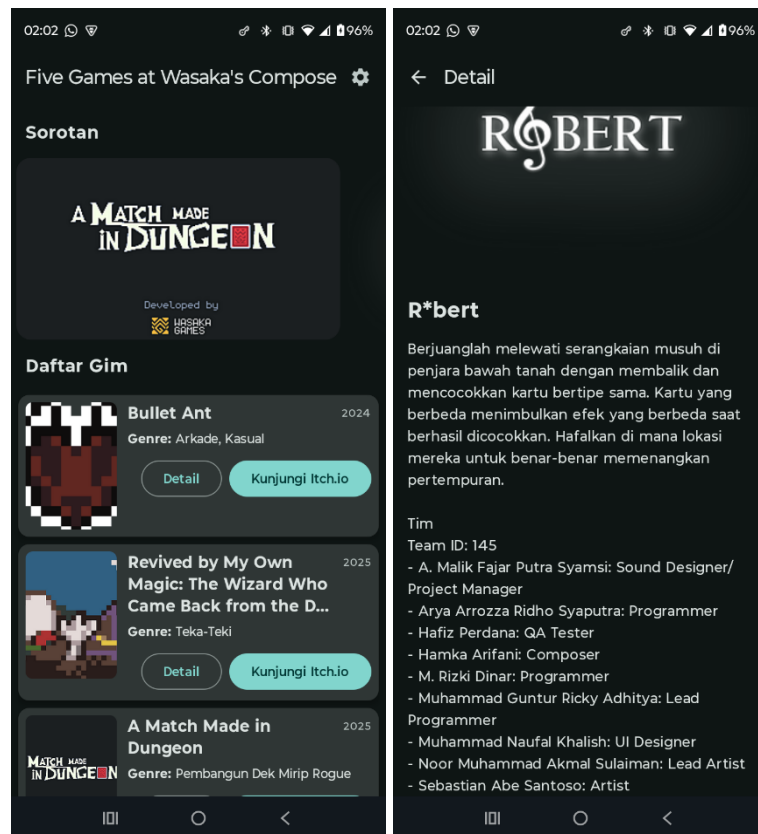
Gambar 4. Screenshot Tampilan Awal Soal 1 Compose



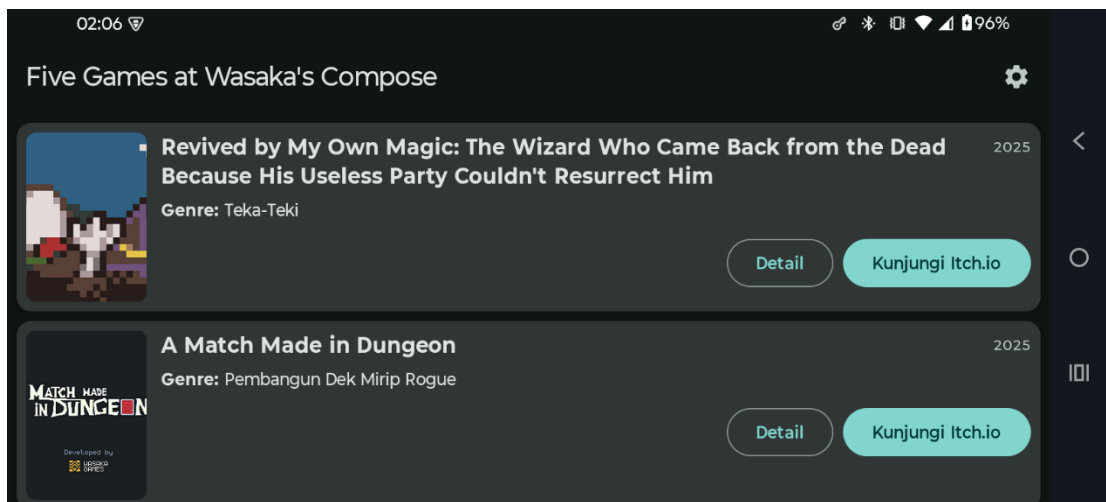
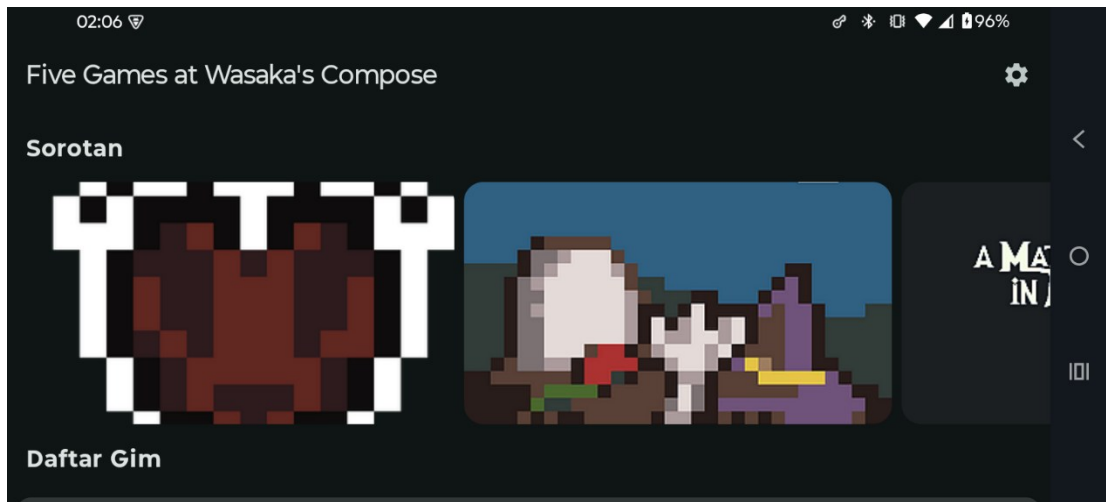
Gambar 5. Screensho Tampilant Detail Soal 1 Compose



Gambar 6. Screenshot Pengaturan Bahasa Soal 1 Compose

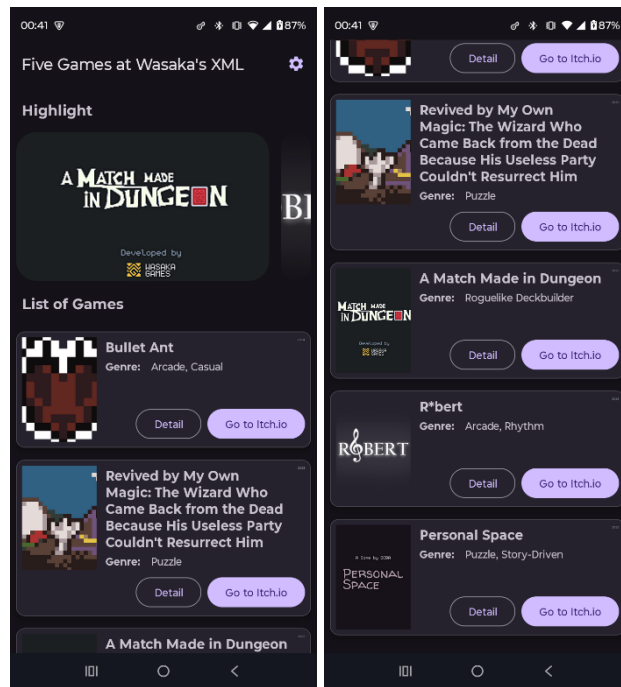


Gambar 7. Screenshot Tampilan Bahasa Indonesia Soal 1 Compose

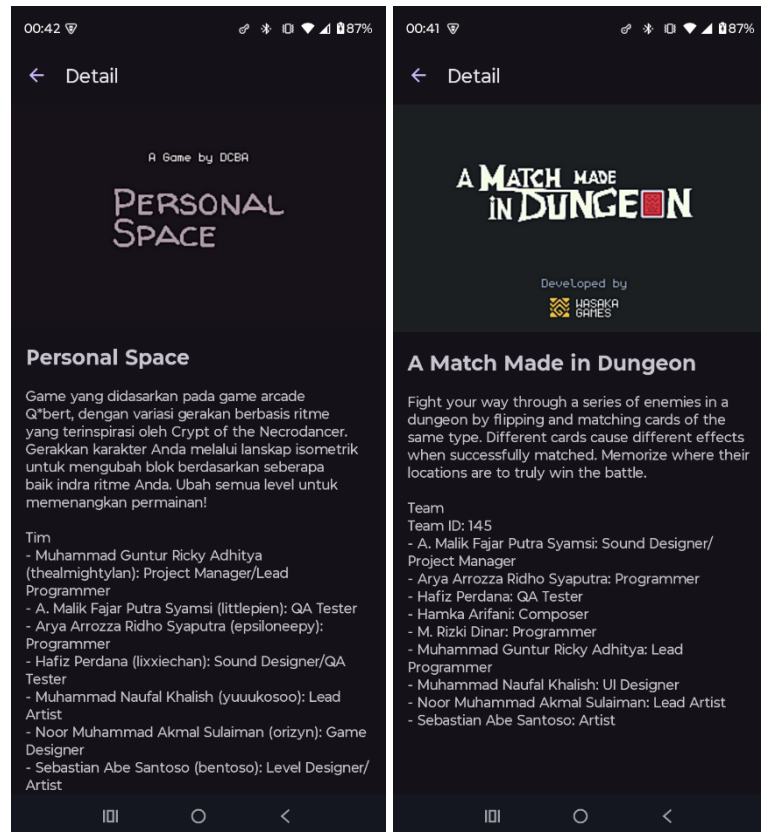


Gambar 8. Screenshot Tampilan Landscape Soal 1 Compose

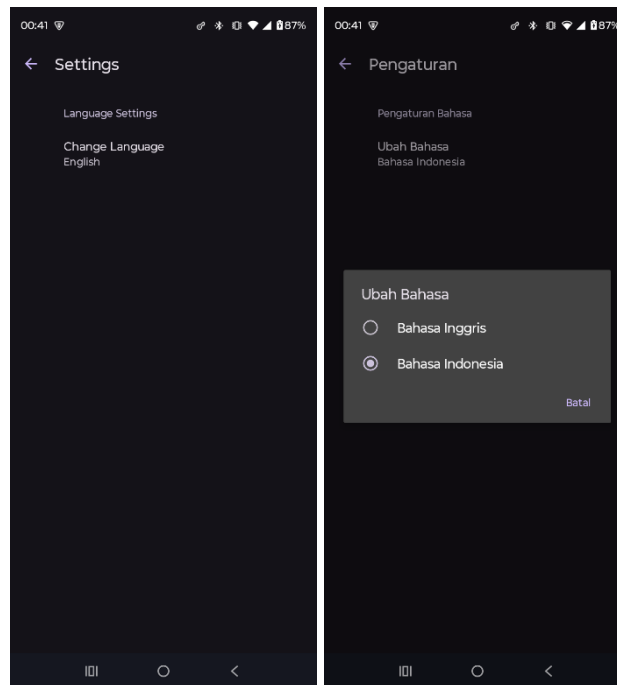
XML



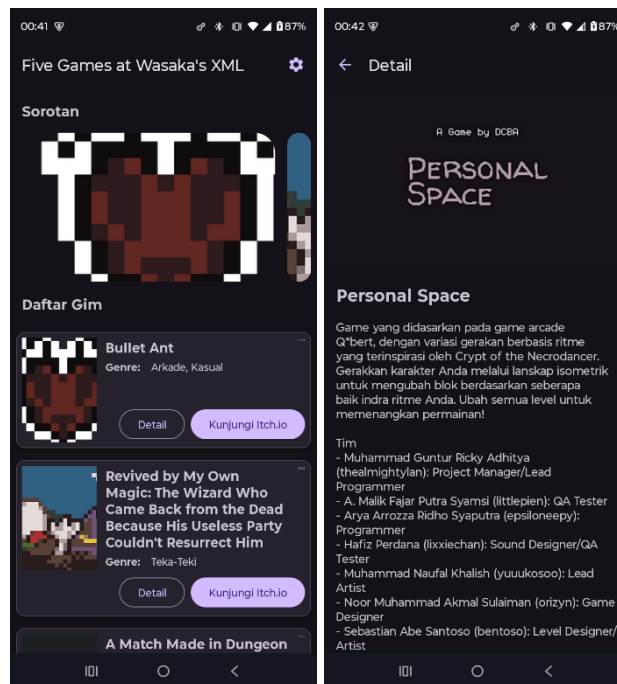
Gambar 9. Screenshot Tampilan Awal Soal 1 XML



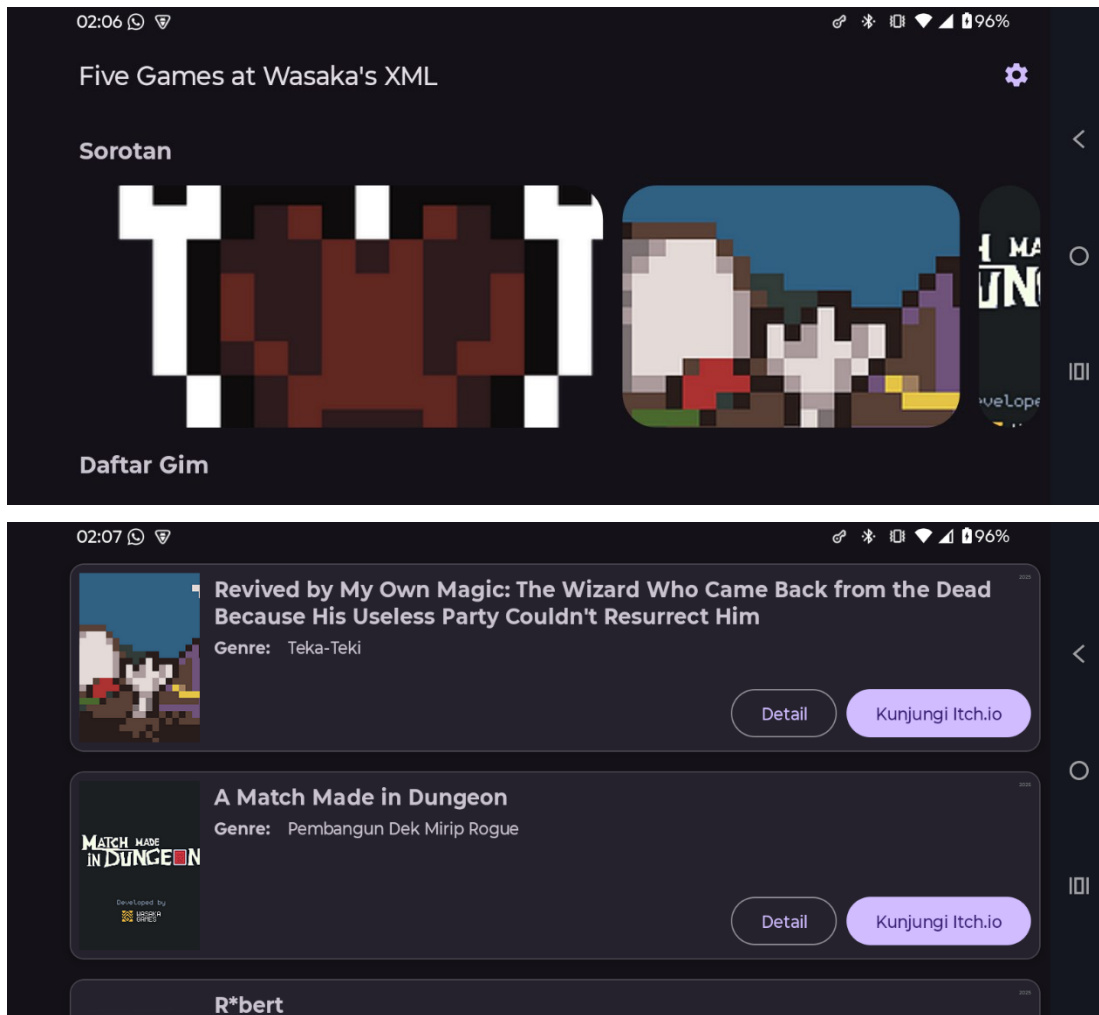
Gambar 10. Screenshot Tampilan Detail Jawaban Soal 1 XML



Gambar 11. Screenshot Pengaturan Bahasa Soal 1 XML



Gambar 12. Screenshot Tampilan Bahasa Indonesia Soal 1 XML



Gambar 13. Screenshot Tampilan Landscape Soal 1 XML

C. Pembahasan

Jetpack Compose

Class MainActivity.kt

Kelas 'MainActivity' adalah titik masuk utama aplikasi yang mewarisi dari 'AppCompatActivity'. Dalam metode 'onCreate()', kelas ini menggunakan 'setContent' untuk membangun antarmuka pengguna dengan Jetpack Compose. Seluruh UI dibungkus dalam tema aplikasi ('FiveGamesComposeTheme'), yang kemudian memanggil composable 'FiveGamesNavGraph' untuk mengatur dan menampilkan seluruh alur navigasi aplikasi.

Class Game.kt

‘Game’ adalah sebuah data class yang berfungsi sebagai model data sederhana untuk merepresentasikan sebuah game. Kelas ini berisi atribut-atribut seperti ‘id’, dan beberapa referensi ke sumber daya Android (‘@DrawableRes’ untuk gambar dan ‘@StringRes’ untuk teks) seperti judul, tahun, genre, detail, dan URL. Penggunaan data class secara otomatis menghasilkan fungsi-fungsi standar seperti equals(), hashCode(), dan toString().

Class GameRepository.kt

Kelas ‘GameRepository’ berfungsi sebagai sumber data tunggal untuk aplikasi ini. Kelas ini memiliki satu metode, ‘getGames()’, yang membuat dan mengembalikan sebuah daftar (list) objek ‘Game’. Data game dalam aplikasi ini bersifat statis (didefinisikan langsung di dalam kode), di mana setiap objek ‘Game’ diinisialisasi dengan data spesifik seperti ID, gambar, dan referensi teks dari sumber daya aplikasi.

Class MainViewModel.kt

Kelas ‘MainViewModel’ bertanggung jawab untuk mengelola dan menyediakan data yang akan ditampilkan di UI, serta bertahan dari perubahan konfigurasi seperti rotasi layar. Kelas ini menggunakan arsitektur MVVM dan StateFlow dari Kotlin Coroutines untuk manajemen state yang reaktif. Di dalamnya, ia membuat instance dari ‘GameRepository’ untuk mendapatkan data game. Data tersebut kemudian disimpan dalam sebuah ‘MutableStateFlow’, yang diekspos sebagai ‘StateFlow’ yang tidak dapat diubah, yang ideal untuk diobservasi oleh UI Compose.

Class HomeScreen.kt

‘HomeScreen’ adalah sebuah composable yang mendefinisikan seluruh UI untuk layar utama aplikasi. Fungsi ini menerima lambda untuk menangani navigasi ke layar detail dan pengaturan. Di dalamnya, ia menggunakan ‘Scaffold’ untuk menyediakan struktur dasar layar, termasuk ‘TopAppBar’ yang berisi judul aplikasi dan sebuah ‘IconButton’ untuk navigasi ke pengaturan. Konten utama layar dibangun menggunakan

‘LazyColumn’, yang secara efisien menampilkan daftar item yang dapat di-scroll. ‘LazyColumn’ ini dibagi menjadi dua bagian utama: sebuah carousel game yang dibuat dengan ‘LazyRow’ (di dalam composable ‘GameCarousel’), dan daftar game vertikal yang dibuat dengan ‘items(games)’. Data game diambil dari ‘MainViewModel’ dan diobservasi menggunakan ‘collectAsState()’, sehingga UI akan diperbarui secara otomatis jika data berubah.

‘GameCarousel’ adalah composable yang menampilkan daftar game secara horizontal. Setiap item di dalamnya adalah sebuah ‘Image’ yang dapat diklik untuk menavigasi ke layar detail.

‘GameListItem’ adalah composable yang mendefinisikan tampilan untuk satu item dalam daftar vertikal. Item ini menggunakan ‘Card’ untuk memberikan tampilan yang menonjol dan ‘Row’ serta ‘Column’ untuk menyusun elemen-elemen seperti gambar, judul, tahun, genre, dan dua buah tombol (‘OutlinedButton’ untuk detail and ‘Button’ untuk membuka link). Logika untuk membuka link eksternal menggunakan ‘Intent’ juga ditangani di sini.

Class DetailScreen.kt

‘DetailScreen’ adalah sebuah composable yang menampilkan detail dari satu game yang dipilih. Fungsi ini menerima ‘gameId’ dan sebuah lambda untuk navigasi kembali. Sama seperti ‘HomeScreen’, ia menggunakan ‘Scaffold’ dengan ‘TopAppBar’ yang memiliki tombol kembali. Data game yang relevan ditemukan dari daftar game yang disediakan oleh ‘MainViewModel’. Konten utama ditampilkan dalam sebuah ‘Column’ yang dapat di-scroll, yang berisi sebuah ‘Image’ besar di bagian atas, diikuti oleh dua buah ‘Text’ untuk menampilkan judul dan deskripsi lengkap dari game tersebut.

Class SettingsScreen.kt

‘SettingsScreen’ adalah sebuah composable yang mendefinisikan UI untuk layar pengaturan. Layar ini menggunakan ‘Scaffold’ dengan ‘TopAppBar’ yang memiliki tombol kembali. Konten utamanya adalah sebuah ‘Column’ yang menampilkan

preferensi pengaturan, dalam hal ini adalah pengaturan bahasa. Terdapat sebuah baris yang dapat diklik untuk membuka dialog pemilihan bahasa. Sebuah state `'showLanguageDialog'` digunakan untuk mengontrol visibilitas dari `'AlertDialog'`. Ketika dialog ditampilkan, ia akan menampilkan daftar bahasa yang tersedia (dalam kasus ini, Inggris dan Indonesia) menggunakan `'RadioButton'`. Ketika pengguna memilih bahasa baru, fungsi ini memanggil `'AppCompatActivity.setApplicationLocales()'` untuk mengubah bahasa aplikasi secara dinamis.

Class Screen.kt

`'Screen'` adalah sebuah sealed class yang digunakan untuk mendefinisikan semua rute navigasi dalam aplikasi secara terpusat dan aman (type-safe). Setiap objek di dalamnya (`'Home'`, `'Detail'`, `'Settings'`) merepresentasikan sebuah layar dan memiliki properti route berupa String. Untuk layar `'Detail'`, yang memerlukan argumen dinamis, terdapat fungsi tambahan `'createRoute()'` untuk membangun rute lengkap dengan ID yang spesifik. Penggunaan sealed class ini membuat kode navigasi lebih bersih dan mengurangi risiko kesalahan pengetikan pada string rute.

Class NavGraph.kt

`'FiveGamesNavGraph'` adalah sebuah composable yang bertanggung jawab untuk mendefinisikan dan mengelola seluruh alur navigasi dalam aplikasi menggunakan Navigation Compose. Di dalamnya, sebuah `'NavHost'` diatur dengan `'navController'` and tujuan awal (`'startDestination'`). Terdapat tiga tujuan yang didefinisikan menggunakan fungsi `'composable'`: satu untuk `'HomeScreen'`, satu untuk `'DetailScreen'`, and satu untuk `'SettingsScreen'`. Rute untuk setiap layar diambil dari sealed class `'Screen'`. Definisi untuk `'DetailScreen'` juga menyertakan deklarasi argument (`'gameId'`) yang menandakan bahwa ia memerlukan sebuah integer saat dinavigasi. Logika navigasi, seperti perpindahan dari `'HomeScreen'` ke `'DetailScreen'` dengan membawa ID, atau navigasi kembali, ditangani dengan memanggil fungsi pada `navController`.

XML

Class MainActivity.kt

Kelas ‘MainActivity’ adalah titik masuk utama aplikasi yang mewarisi dari ‘AppCompatActivity’. Dalam metode ‘onCreate()’, kelas ini mengaktifkan tampilan edge-to-edge dan menggunakan View Binding (‘ActivityMainBinding’) untuk menghubungkan layout XML ke kode. Ini adalah praktik modern untuk mengakses elemen UI secara aman dan efisien tanpa perlu menggunakan ‘findViewById’.

Class Game.kt

‘Game’ adalah sebuah data class yang berfungsi sebagai model data sederhana untuk merepresentasikan sebuah game. Kelas ini berisi atribut-atribut seperti ‘id’, dan beberapa referensi ke sumber daya Android (‘@DrawableRes’ untuk gambar dan ‘@StringRes’ untuk teks) seperti judul, tahun, genre, detail, dan URL. Penggunaan data class secara otomatis menghasilkan fungsi-fungsi standar seperti equals(), hashCode(), dan toString().

Class GameRepository.kt

Kelas ‘GameRepository’ berfungsi sebagai sumber data tunggal (single source of truth) untuk aplikasi ini. Kelas ini memiliki satu metode, ‘getGames()’, yang membuat dan mengembalikan sebuah daftar (list) objek ‘Game’. Data game dalam aplikasi ini bersifat statis (didefinisikan langsung di dalam kode), di mana setiap objek ‘Game’ diinisialisasi dengan data spesifik seperti ID, gambar, dan referensi teks dari sumber daya aplikasi.

Class MainViewModel.kt

Kelas ‘MainViewModel’ bertanggung jawab untuk mengelola dan menyediakan data yang akan ditampilkan di UI, serta bertahan dari perubahan konfigurasi seperti rotasi layar. Kelas ini menggunakan arsitektur MVVM. Di dalamnya, ia membuat instance dari ‘GameRepository’ untuk mendapatkan data game. Data tersebut kemudian

disimpan dalam sebuah ‘MutableLiveData’, yang diekspos sebagai ‘LiveData’ yang tidak dapat diubah. Blok ‘init’ memastikan bahwa data game langsung dimuat saat ‘ViewModel’ pertama kali dibuat.

Class HomeFragment.kt

Kelas ‘HomeFragment’ adalah komponen UI yang menampilkan layar utama aplikasi. Kelas ini menggunakan View Binding (‘FragmentHomeBinding’) untuk mengakses elemen-elemen layoutnya. Di dalam metode ‘onViewCreated()’, fragment ini menginisialisasi dan mengatur dua buah ‘RecyclerView’: satu untuk tampilan carousel dan satu lagi untuk daftar game vertikal. Fragment ini juga mengobservasi data dari ‘MainViewModel’; setiap kali data game berubah, adapter dari kedua ‘RecyclerView’ akan diperbarui secara otomatis. Terdapat juga logika untuk menangani klik pada tombol pengaturan (yang akan menavigasi ke ‘SettingsFragment’), serta metode-metode privat untuk mengatur adapter, ‘RecyclerView’, dan navigasi ke halaman detail dengan membawa ID game.

Class DetailFragment.kt

Kelas ‘DetailFragment’ bertanggung jawab untuk menampilkan detail dari satu game yang dipilih. Kelas ini juga menggunakan View Binding (‘FragmentDetailBinding’). Dalam metode ‘onViewCreated()’, ia mengambil ‘gameId’ yang dikirimkan melalui arguments dari fragment sebelumnya. Dengan ID tersebut, ia mencari objek ‘Game’ yang sesuai dari ‘GameRepository’. Jika game ditemukan, fragment ini akan mengisi elemen-elemen UI seperti ‘ImageView’ dan ‘TextView’ dengan data dari objek game tersebut. Terdapat juga listener pada toolbar yang memungkinkan pengguna untuk kembali ke layar sebelumnya.

Class SettingsFragment.kt

Kelas ‘SettingsFragment’ adalah layar pengaturan yang mewarisi dari ‘PreferenceFragmentCompat’, sebuah kelas standar Android untuk membuat halaman pengaturan. Dalam metode ‘onCreatePreferences()’, fragment ini memuat hierarki

pengaturan dari file XML ('R.xml.root_preferences'). Ia juga mencari preferensi bahasa ('ListPreference') dan memasang listener padanya. Ketika pengguna memilih bahasa baru, listener tersebut akan dieksekusi untuk mengubah lokal aplikasi menggunakan 'AppCompatActivity.setApplicationLocales()'. Metode 'onCreateView()' dan 'onViewCreated()' di-override untuk mengintegrasikan layout preferensi ke dalam layout custom yang memiliki toolbar dengan tombol kembali.

Class GameAdapter.kt

Kelas 'GameAdapter' adalah sebuah 'ListAdapter' yang dirancang untuk 'RecyclerView' yang menampilkan daftar game secara vertikal. Kelas ini menggunakan View Binding ('ItemGameVerticalBinding') dan 'DiffUtil' untuk performa yang efisien saat memperbarui daftar. Constructor-nya menerima dua fungsi lambda sebagai callback untuk menangani klik pada tombol detail dan tombol link. Di dalam 'GameViewHolder', metode 'bind()' bertugas untuk mengisi data dari objek 'Game' ke elemen-elemen UI yang sesuai pada setiap item, serta memasang listener pada kedua tombol tersebut.

Class CarouselAdapter.kt

Kelas 'CarouselAdapter' adalah 'ListAdapter' yang dirancang khusus untuk 'RecyclerView' yang berfungsi sebagai carousel. Mirip dengan 'GameAdapter', kelas ini juga menggunakan View Binding ('ItemGameCarouselBinding') dan 'DiffUtil' (memanfaatkan 'DiffCallback' yang sama dari 'GameAdapter'). Constructor-nya menerima satu fungsi lambda sebagai callback untuk menangani klik pada gambar. Dalam metode 'onBindViewHolder()', ia mengatur gambar untuk setiap item di carousel dan memasang listener klik pada gambar tersebut.

File activity_main.xml

File XML ini adalah layout utama untuk 'MainActivity'. Di dalamnya hanya terdapat satu elemen, yaitu '<androidx.fragment.app.FragmentContainerView>'. Komponen ini berfungsi sebagai wadah untuk menampung dan mengelola navigasi antar fragment.

Atribut `'app:navGraph="@navigation/nav_graph"'` menautkannya ke grafik navigasi, dan `'app:defaultNavHost="true"'` menjadikannya sebagai host navigasi utama yang akan menangani tombol kembali sistem.

File fragment_home.xml

File XML ini mendefinisikan tata letak untuk 'HomeFragment'. Tag akarnya adalah `'<androidx.core.widget.NestedScrollView>'`, yang memungkinkan seluruh konten di dalamnya untuk di-scroll. Di dalamnya, sebuah `'<LinearLayout>'` vertikal menyusun semua elemen: sebuah `'<com.google.android.material.appbar.MaterialToolbar>'` sebagai app bar yang berisi judul dan sebuah `'<ImageButton>'` untuk navigasi ke pengaturan; dua buah `'<TextView>'` yang berfungsi sebagai judul untuk bagian highlight dan daftar game; serta dua buah `'<androidx.recyclerview.widget.RecyclerView>'`, satu dengan ID `'rv_carousel'` untuk tampilan carousel dan satu lagi dengan ID `'rv_games'` untuk daftar vertikal.

File fragment_detail.xml

File XML ini mendefinisikan tata letak untuk 'DetailFragment'. Tag akarnya adalah `'<LinearLayout>'` vertikal yang berisi sebuah `'<com.google.android.material.appbar.MaterialToolbar>'` di bagian atas dengan ikon kembali. Di bawahnya, sebuah `'<androidx.core.widget.NestedScrollView>'` membungkus sebuah `'<androidx.constraintlayout.widget.ConstraintLayout>'`. Di dalam 'ConstraintLayout' ini, terdapat sebuah `'<ImageView>'` untuk menampilkan gambar detail game, dan dua buah `'<TextView>'` di bawahnya untuk menampilkan judul dan deskripsi lengkap dari game tersebut.

File fragment_settings.xml

File XML ini mendefinisikan tata letak untuk 'SettingsFragment'. Tag akarnya adalah `'<LinearLayout>'` vertikal yang berisi sebuah `'<com.google.android.material.appbar.MaterialToolbar>'` di bagian atas dengan ikon kembali dan judul "Settings". Di bawahnya, terdapat sebuah `'<FrameLayout>'` dengan ID `'settings_container'`.

‘FrameLayout’ ini berfungsi sebagai wadah di mana konten dari ‘PreferenceFragmentCompat’ akan disisipkan secara dinamis oleh kode Kotlin.

File item_game_vertical.xml

File XML ini mendefinisikan tata letak untuk satu item dalam ‘RecyclerView’ vertikal. Tag akarnya adalah ‘<com.google.android.material.card.MaterialCardView>’, yang memberikan tampilan kartu dengan sudut membulat dan bayangan. Di dalamnya, sebuah ‘<androidx.constraintlayout.widget.ConstraintLayout>’ digunakan untuk menyusun elemen-elemen seperti ‘<ImageView>’ untuk gambar game, beberapa ‘<TextView>’ untuk menampilkan judul, tahun, dan genre, serta sebuah ‘<LinearLayout>’ horizontal di bagian bawah yang berisi dua buah ‘<Button>’ untuk “Detail” dan “Link”.

File item_game_carousel.xml

File XML ini mendefinisikan tata letak untuk satu item dalam ‘RecyclerView’ carousel. Tag akarnya adalah ‘<com.google.android.material.carousel.MaskableFrameLayout>’, sebuah komponen khusus untuk carousel yang memungkinkan penerapan mask atau bentuk pada kontennya (misalnya, sudut membulat). Di dalamnya hanya terdapat satu ‘<ImageView>’ yang mengisi seluruh area frame dan akan menampilkan gambar game.

File nav_graph.xml

File XML ini adalah grafik navigasi yang mendefinisikan alur perpindahan antar fragment dalam aplikasi menggunakan Navigation Component. Tag akar ‘<navigation>’ menetapkan ‘homeFragment’ sebagai tujuan awal (app:startDestination). Terdapat tiga tag ‘<fragment>’ yang mendefinisikan setiap tujuan navigasi: ‘homeFragment’, ‘detailFragment’, dan ‘settingsFragment’. Di dalam ‘homeFragment’, terdapat dua tag ‘<action>’ yang mendefinisikan transisi yang mungkin dari layar utama, yaitu ke ‘detailFragment’ dan ke ‘settingsFragment’. ‘detailFragment’ juga mendefinisikan sebuah ‘<argument>’ bernama “gameId”, yang menandakan bahwa ia mengharapkan sebuah data integer saat dinavigasi.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

Karena LazyColumn tidak ada di XML. Sebenarnya sesederhana itu. Kebanyakan proyek Android masih menggunakan XML.

Namun, seandainya ada LazyColumn di XML, menurut saya RecyclerView masih sangat relevan karena sejatinya komponen Android Views (XML) akan jauh lebih cepat dan efisien daripada komponen Jetpack Compose. Meskipun LazyColumn menawarkan gaya penulisan kode yang jauh lebih ringkas dan modern, RecyclerView dipertahankan penggunaannya karena alasan performa mentah serta efisiensi sumber daya. Acap kali orang-orang di kalangan developer mengatakan bahwa proyek mereka yang dibuat dengan LazyColumn/LazyRow lebih lambat daripada yang dibuat dengan RecyclerView.

Selain urusan performa, Android Views dan RecyclerView sesungguhnya sudah cukup mumpuni dan dibekali dengan berbagai kemampuan kustomisasi yang sangat spesifik. Walaupun penulisan kode dalam Jetpack Compose jauh lebih bersih, penanganan kasus-kasus seperti tata letak yang di luar standar justru sering kali lebih mudah dan lebih stabil ditangani menggunakan arsitektur bawaan RecyclerView yang memang dirancang untuk dimanipulasi secara manual.

Faktor keberlanjutan bisnis dan tingginya risiko migrasi juga menjadi alasan terbesar mengapa kode yang panjang dan bersifat boilerplate ini tetap dipertahankan oleh banyak perusahaan teknologi. Mengganti keseluruhan arsitektur antarmuka aplikasi dari RecyclerView yang sudah berjalan sangat stabil dan bebas hambatan ke dalam teknologi Jetpack Compose akan memakan banyak waktu, tenaga, dan biaya pengujian ulang. Karena sistem manajemen memori dari RecyclerView sudah terbukti mampu diandalkan untuk menangani aplikasi berskala jutaan pengguna tanpa masalah kelambatan, tim pengembang umumnya memilih untuk mempertahankan fondasi yang

sudah solid tersebut. Pendekatan yang paling sering dilakukan di industri saat ini adalah menggunakan antarmuka modern LazyColumn hanya pada pembuatan halaman aplikasi yang benar-benar baru, sementara sistem daftar utama yang sudah ada sebelumnya tetap membiarkan RecyclerView bekerja di balik layar.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

A. Pembahasan

Menurut saya, bukan penerapan Clean Architecture buatan Uncle Bob itu sendiri yang diharuskan, melainkan penerapan arsitektur yang bagus dalam pengembangan proyek, baik proyek berbahasa Kotlin maupun bahasa lain. Ada banyak arsitektur yang secara konsep menyerupai Clean Architecture, dan arsitektur-arsitektur itu pun bisa digunakan secara bersamaan.

Layaknya arsitektur MVC dan MVVM, Clean Architecture membagi proyek ke dalam beberapa lapisan. Dengan demikian, sistem yang dibangun dapat beradaptasi seiring perkembangan zaman. Apabila di masa depan tim developer memutuskan untuk mengganti teknologi tampilan dari sistem Android Views (XML) menjadi Jetpack Compose, atau bermigrasi dari basis data lokal ke sistem penyimpanan awan yang berbeda, lapisan domain yang berisi aturan business logic sama sekali tidak perlu disentuh atau ditulis ulang karena ia tidak memiliki ketergantungan pada komponen eksternal tersebut.

Skalabilitas dan kemudahan dalam kolaborasi tim juga menjadi alasan utama mengapa arsitektur-arsitektur ini sangat direkomendasikan untuk proyek perangkat lunak modern. Seorang developer dapat fokus merangkai UI di sisi frontend, sementara developer lain dapat membangun logika server di sisi backend.

Terakhir, pengujian bisa dipermudah dengan penerapan Clean Architecture. Pengujian unit dapat dieksekusi dalam hitungan detik secara lokal tanpa memerlukan proses peluncuran emulator atau perangkat fisik yang lambat. Developer dapat memvalidasi semua skenario kasus penggunaan aplikasi dengan tingkat cakupan kode yang sangat tinggi, yang pada akhirnya akan menekan jumlah masalah kritis yang mungkin muncul saat aplikasi sudah dirilis.

TAUTAN GIT

<https://github.com/orizynpx/praktikum-pemrograman-mobile/tree/main/Modul%203>